

Mitigating Log Poisoning Attack against Anomaly-based Intrusion Detection System

Alexandre Azor^{1,2}, Françoise Sailhan², Alexandre Reiffers-Masson², and Julien Francq³

¹ Chaire de cybergdéfense des systèmes navals, Brest, France

² IMT Atlantique, Brest, France

³ Naval Group (CERT), Ollioules, France

Abstract. Anomaly and Intrusion Detection Systems are threatened by poisoning attacks that alter the behaviour of classifiers. Poisoning attacks carefully corrupt training samples and the learnt model, thereby rendering the detection ineffective. To mitigate poisoning attacks, we propose an ensemble learning method that trains several classifiers on distinct portions of the training set. At runtime, the detection system aggregates the classification results. Our theoretical analysis shows that detection is robust to poisoning attacks, when the number of poisoned samples is bounded, which is the case in practice to keep the attacks stealth. To support empirical evaluations of the robustness, we designed and implemented several types of poisoning attacks and developed a prototype. Evaluations show that our mitigation approach significantly improves the robustness and accuracy of the detection system compared to vanilla detection systems.

Keywords: Log anomaly detection · Log poisoning attack · Mitigation

1 Introduction

Information systems are constantly under attack [24], it is henceforth critical to look for early signs of intrusions into the information system, which result in unexpected or abnormal behaviour. In this regard, *Anomaly-based Intrusion Detection Systems* (AIDSs) [15] model the normal behaviour of the information system and raise an alarm when the observed behaviour differs from the model depicting that normal behaviour. To this end, AIDSs use the computer logs that depict the state of the information system and detect anomalies relying on a variety of Machine Learning (ML) or Deep Learning (DL) algorithms, including the Long Short-Term Memory neural network (LSTM) [12]. Despite their ability to classify logs, with a high accuracy and F1-score, AIDSs are vulnerable to poisoning attacks. If the training set is altered (i.e., poisoned) then the AIDS's ability to correctly detect anomalies will also be affected. As stated in [30], poisoning attacks are a serious threat for entities using ML or DL tools. To the best of our knowledge, in most of the research on the poisoning of training data for ML or DL algorithms designed for text classification, non-targeted poisoning

is not considered [4,23] or specifically concerns certain ML algorithms [16,17]. The same applies for methods that improve the robustness against training data poisoning: [22,5,32,13] do not account for non-targeted poisoning, while [25] is specific to Principal Component Analysis (PCA). In complement to the above effort, we introduce a mitigation technique that reduces the impact of untargeted log poisoning against DL classifiers that sustain AIDSs. Our aim is to develop an AIDS that mitigates log poisoning attacks, taking advantage of the fact that log poisoning is sparse to remain undetectable. Circumventing training data poisoning is essential for accurate classification: with 3% of the training set poisoned, [17] shows that the test errors of a Support-Vector Machine (SVM)-based classifier increases from 3% to 23%. Our mitigation solution is based on an ensemble learning method: the AIDS essentially combines several DL classifiers together. Our classifiers are based on LSTM models. Beforehand, classifiers are trained on some distinct parts of the training set. Then, the AIDS aggregates the classifiers to essentially remove the faults done by individual classifier(s), and therefore reduces detection error. Going one step further, we combine ensemble learning and DL techniques, leveraging a median method. We compare the combined model obtained via the median method with an aggregate model in which the prediction is based on a majority vote performed by the ensemble of classifiers. Provided that enough base models and a suitable method to allocate training samples, experiments show that the combined model trained on a poisoned dataset exhibits similar performance in terms of F1-score to that obtained when trained on a clean dataset. In addition, we present a rigorous analysis of the robustness of our aggregation methods. In summary, our key contributions are the following:

- we present related work on log-based anomaly detection, with a focus on their ability to detect and mitigate poisoning attacks (Section 2);
- we characterise and formulate log poisoning attacks and the countermeasures that sustain robust detection (Section 3);
- we propose a theoretical analysis on the robustness of the detection system (Section 4);
- we implement two log poisoning attacks that incorporate corrupted log sequences to the training set; poisoned sequences either randomly spread or tend to be very close to each other;
- we present a prototype integrated to a log anomaly detector based on Deep-Log [6]. Our prototype sustains a mitigation method alleviating log poisoning. In particular, we investigate two methods that intelligently partition the training dataset and a family of ensemble learning methods, whose performance are further empirically evaluated (Section 5).

2 Background

We present the main categories of attacks that target log-based anomaly detectors (§ 2.1) and the methods to mitigate those attacks (§ 2.2).

2.1 Attacks targeting log anomaly detectors

With the wide adoption of DL and ML algorithms by a wide range of critical applications including AIDSs, attackers are strongly incentivized to manipulate the datasets, the models and results of the AIDS.

(Untargeted) Poisoning attacks [7,9] tamper training data by adding malicious entries with the aim of diminishing the ability of the anomaly detector to accurately detect anomalies. In [30], the authors identify high-confidence samples labelled as benign in the training set, and craft poisoned samples that are close to each other in the feature space but that are labelled as anomalies in order to compromise the SVM-based network anomaly detector. In [16], attacker injects carefully selected vectors into the training set. The aim is to shift in a desired direction, the set of vectors considered as legitimate by the target model corresponding to a centroid anomaly detector operating on network traffic. Experiments show that an attacker needs substantial resources to impact the anomaly detector. In [8], an analysis is provided about the errors in the training samples that lead to parsing errors with the log-based anomaly detectors. In [17], the attacker crafts poisoned samples that evade mitigation methods (which remove such samples) and that compromise SVM-based text classifiers. Crafting a sufficient amount of poisoned samples involves a significant computational cost and takes around one hour.

Backdoor attack [20] decreases the detector performance for specific log entries. The attacker injects malicious entries in the training set so that the behaviour of the poisoned model is altered only for the entries that contain a *trigger*. With DL models operating on textual data, the *trigger* can be the log entries that constitute the beginning and the end of a sequence of logs [2], a sentence [4] or a turn of phrase [23]. The goal of backdoor attacks is that malicious entries containing the *trigger* are misclassified as benign during the inference.

Contrary to poisoning and backdoor attacks, *adversarial attacks* take place during the inference. Inputs are modified in a subtle way so that the model behaves very differently. Adversarial attacks against DL classifiers that process textual data, modify the input as little as possible by altering and deleting words, or reversing their order. As a result, altered input is misclassified [27,19,14,28]. An adversarial attack, which targets anomaly detector that labels a malicious input as benign, is called an *evasion attack* [21,11].

In this article, we present a poisoning attack targeting a log-based anomaly detector that uses a deep neural network.

2.2 Countermeasures against attacks

To mitigate the impact of poisoning attacks, one may identify and remove compromised samples [30], use robust statistical methods [25] or correct corrupted samples [8]. In order to sanitise the training set, [30] computes an anomaly score to each training sample and stores the samples with the lowest score. The method requires a clean validation set, which reduces the application usability. [25] leverages robust statistics to limit the impact of poisoning on an anomaly

detector using PCA-based network. However, the method is not easily applied to algorithms other than PCA. In [8], the framework for robust learning deals with parsing errors without relying on a specific ML or DL algorithm. Leveraging Natural Language Processing (NLP), the system merges log templates associated to similar log messages and hence mitigates the impact of parsing errors. However, a complete list of the existing log templates is required.

To handle backdoor, some methods [5,32,13] prevent the model from learning associations between inputs containing a *trigger* and a specific output. In particular, methods add constraints and noise to the gradient using e.g., *Differential Privacy*. Robustness against adversarial attacks is achieved either through the use of *adversarial training* [22], that is, by training a model that handles tampered inputs, or by making the preprocessing and analysis robust against input tampering [31,29].

3 Mitigating Log Poisoning

We introduce a robust anomaly detection framework that deals with log poisoning attacks: multiple classifiers are trained on subsets of the original training dataset. Then, the predictions of each model are combined through different aggregation schemes that detect anomalous sequences of events. Our mitigation method against poisoning attacks is based on ensemble learning [26].

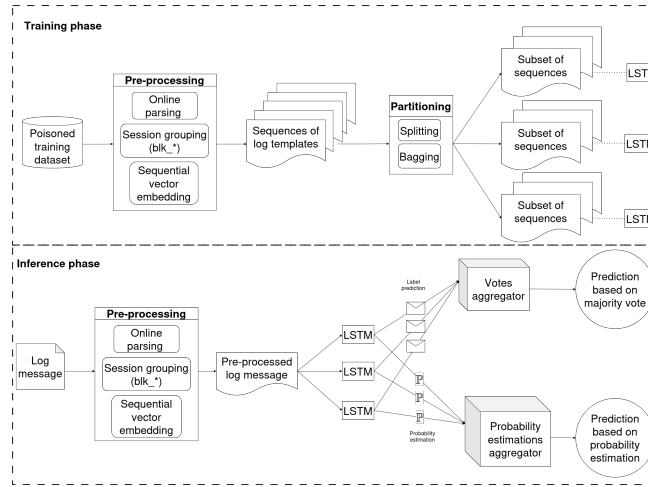


Fig. 1: Workflow of the anomaly detector

Our AIDS preprocesses the logs (Figure 1) as follows: logs are sequentially parsed, i.e., the log entries (e.g. timestamps, IP addresses, resource identifiers) that are variable are removed. The resulting string is called a *log template*. Each log template is associated to an identifier. The set of identifiers is denoted K

Notation	Description
K	Set of identifiers of log templates
$\mathcal{S}_{train}$	Log sequences in the training set
$\tilde{\mathcal{S}}$	Poisoned log sequences added to the training set
$\mathcal{S}_r$	Legitimate benign training log sequences removed by the attacker
$\mathcal{S}'_{train}$	Poisoned training set
N_{train}	Number of training sequences
$N_{mislabelled}$	Number of poisoned sequences in the training set
N_{models}	Number of base models used by the mitigation method
δ	Ratio of poisoned sequences in the training set
$\mathcal{Y}$	Set of labels
W	Burst window size
γ	Size of the burst window divided by the size of the training set
$\hat{p}(a w)$	Estimation made by the model of the probability that a log template a comes right after an observation window w

Table 1: Notations

and satisfies $K = \{0, \dots, |K| - 1\}$. Templates are grouped together according to a common feature (corresponding to e.g. a block or session ID) and form sequences ordered in chronological order that are embedded into a vector using one-hot encoding. The sequence of vectors is then analysed by the AIDS to detect anomalous behaviours. As illustration, a sequence of log templates such as “Failed connection attempt by user $\langle * \rangle$ ” $\rightarrow$ “User $\langle * \rangle$ successfully logged in” $\rightarrow$ “User $\langle * \rangle$ logs out” is mapped to a finite alphabet as $a \rightarrow b \rightarrow c$, where each symbol identifies a log template. This sequence is classified as normal, because a user entering a password often makes a single mistake. In contrast, a sequence such as $a \rightarrow a \rightarrow a \rightarrow a$ is labelled as anomalous given that a series of unsuccessful login attempts is likely a sign of a brute-force attack. The training set $\mathcal{S}_{train}$ consists of finite and ordered sequences (following a chronological order of their first occurrences) of log templates characterised by varying sizes. Each sequence in the dataset is associated to a label $y \in \mathcal{Y} := \{-1, 1\}$, where $y = 1$ denotes an anomalous sequence and $y = -1$ a benign sequence.

3.1 Log Poisoning Attack

The attacker accesses the training set $\mathcal{S}_{train}$ and replaces sequences of log entries. With our anomaly detector, the training set is only composed of benign sequences and we suppose that the attacker is discreet and performs a very limited amount of modifications. The poisoning attack replaces some legitimate benign sequences of $\mathcal{S}_{train}$ by anomalous sequences. The resulting poisoned training set $\mathcal{S}'_{train}$ satisfies: $\mathcal{S}'_{train} = (\mathcal{S}_{train} \setminus \mathcal{S}_r) \cup \tilde{\mathcal{S}}$ with $\tilde{\mathcal{S}}$ denoting the set of anomalous sequences and $\mathcal{S}_r \subset \mathcal{S}_{train}$ denoting the set of all the legitimate sequences that are replaced. We define the two following poisoning strategies:

- **Diffuse attack** consists in uniformly and randomly corrupting without replacement $N_{mislabelled}$ sequences of the log template from $\mathcal{S}_{train}$. with The

aim of the diffuse attack is to hide punctual intrusions. The method has been chosen to capture a contamination which is not localised.

- **Burst attack** consists in modifying $N_{\text{mislabelled}}$ sequences of log templates that are close to each other in $\mathcal{S}_{\text{train}}$, i.e., that belong to a burst window W of contiguous sequences of $\mathcal{S}_{\text{train}}$ with $|W| \ll N_{\text{train}}$. The burst attack aims at hiding intensive attacks (e.g., denial of service or brute-force attacks). Burst attacks randomly and uniformly select $N_{\text{mislabelled}}$ sequences of log templates without replacement from the burst window. Let us illustrate burst attacks in which a hacker exfiltrates sensitive data from a system as quickly as possible⁴. This attack induces a high amount of anomalous activities in a short period of time and to reduce the ability of the anomaly detector to spot the attack, the attacker injects in the training set some poisoned logs maliciously labelled as benign and corresponding to the observations made under such an attack.

3.2 Anomaly Detection

Our AIDS attempts to determine whether a sequence of log templates $s = (s_1, \dots, s_l)$ is normal or anomalous. For this purpose, our anomaly detection system relies on DeepLog [6] that preprocesses logs and our AIDS successively predicts the probability of the next log template, based on the h preceding log templates. The subsequence of h preceding log templates is defined as an *observation window* and modelled as an homogeneous Markov chain whose states correspond to the observation windows. Precisely, a model successively analyses and slides the observation window $(s_t, \dots, s_{t+h-1})$ and estimates the distribution of the next log template s_{t+h} based on the previous observation window. For the log template $a \in K$, $p(a|w)$ denotes the probability to observe the log template a right after the sub-sequence $w = (w_1, \dots, w_h)$ of log templates and satisfies:

$$p(a|w) := \mathbb{P}(s_{t+h} = a | s_t = w_1, \dots, s_{t+h-1} = w_h) \quad (1)$$

for any $t \in \{1, \dots, l(s) - h\}$. A parametrized model (i.e., a LSTM with DeepLog) learns $p(a|w)$ for any $a \in K$ and $w \in K^h$. Let $\hat{p}(a|w)$ denote the estimation made by the model. From these probabilities estimations, the AIDS considers that an observed log template a is normal if a belongs to the g most probable log templates knowing w . Otherwise, a is considered anomalous. Finally, a sequence s is flagged as anomalous if and only if an anomalous log template is detected when analysing the observation windows.

Overall, the ability of the AIDS to provide precise estimations of $p(a|w)$ and therefore accurate prediction is challenged by poisoning attacks (Section 3.1). In the following section, we present the mitigation method integrated to the AIDS.

⁴ <https://blog.barracuda.com/2024/06/11/Dwell-time-declining>. Last accessed: 2026-06-03

3.3 Mitigation methods

Our mitigation method relies on an ensemble of base models that are independently trained on distinct portions of the training dataset. At inference time, the detector interrogates the base models, aggregates their predictions and produces a final prediction. By partitioning the training data and aggregating multiple models, the influence of corrupted subsets is mitigated by the final detector. We propose the two following approaches that select which instances of the training set are provided to each classifier:

- **Splitting strategy** divides the (poisoned) training set into disjointed subsets having the same size. Using the same DL-based algorithm, each base model is trained on a different subset. Thus, poisoned logs are likely affecting only a couple of models and not the final detector. The sequences of log templates are ordered and the training of base models training sets is obtained by grouping q_s consecutive sequences, with $q_s := \lfloor \frac{\text{Training set size}}{\text{Number of base models}} \rfloor$.
- **Bagging** [1] consists in uniformly sampling with replacement elements from the original poisoned training set to build new training sets for the base models. The size of the training sets q_b is chosen by the operator. The bagging strategy is likely to generate training sets such that most of them are not as poisoned as the original training set. It is a standard method to improve robustness against poisoning attacks [3].

During the inference process, sequences of log templates are provided to all the base models and each model produces a classification output. The AIDS aggregates the classification outputs using the following aggregation methods:

- **Majority vote**: The AIDS decision is based on the label predicted by the majority of the base models, which is more robust to poisoning attacks compared to a single model trained on the full dataset. For instance, if three models output the labels $(1, 1, -1)$ for a given sequence, the final decision supported by the majority is 1.
- **Robust probability estimation based on the median**: recall that for a given observation window, each model predicts a probability for each possible next log event. For example, for three events a_1, a_2, a_3 : model 1 gives $(0.8, 0.1, 0.1)$, model 2 gives $(0.7, 0.2, 0.1)$ and model 3 gives $(0.1, 0.3, 0.6)$. The median aggregation works as follows: the system aggregates the predictions coordinate-wise by taking the median of each event, yielding $(0.7, 0.2, 0.1)$. Finally, the system then uses this new predictor to flag anomalous sequences.

4 Theoretical analysis

We analyse the majority vote (Section 4.1), the robust estimation (Section 4.2) and the effect of the hyperparameter g on the robustness of the anomaly detection method (Section 4.3).

4.1 Majority vote

We provide an upper bound on the false positive rate considering the majority vote method under a simple statistical model. Table 2 summarizes the notations. We assume that all log sequences have the same length l and are mutually independent. The AIDS consists of N_{models} base models. Each base model is indexed by $m \in \{1, \dots, N_{models}\}$ and observes d independent log sequences $X^{(m,1)}, \dots, X^{(m,d)}$, where $X^{(m,j)} = (X_1^{(m,j)}, \dots, X_l^{(m,j)})$ is the j -th log sequence observed by model m . We assume that a small fraction of the observed sequences are poisoned and that these sequences are distributed randomly among the base models. We further assume that there exists a threshold on the proportion of poisoned sequences in a model's training set below which the model's behaviour remains unchanged. This assumption is discussed further in Section 4.3. The unknown set of base models affected by the poisoning attack is denoted by $\mathcal{C} \subset \{1, \dots, N\}$, $|\mathcal{C}| = \nu N_{models}$, with $0 \leq \nu \leq 1$. We assume that there exists a benign distribution P over K such that, for every model whose index is not in $\mathcal{C}$, the conditional distribution introduced in Equation 1 satisfies $p(\cdot|w) = P(\cdot)$ for every subsequence w . Similarly, we assume that there exists a distribution Q describing the conditional distribution associated with the affected base models. Based on the d observed sequences, each model m constructs an empirical estimation of P if $m \notin \mathcal{C}$, or of Q if $m \in \mathcal{C}$.

For $m \in \{1, \dots, N_{models}\}$ and a sequence of logkeys X^{test} of size l , the model m considers X^{test} as benign if and only if each logkey within X^{test} is among the g most probable logkeys according to $\hat{R}_m$. We assume that X^{test} is independent from the test sequences $\{X^{(m,j)}\}$ and that $S_{m,g}$ is defined with no ambiguity. Let $S_{m,g}$ be the set of the g logkeys of K with the highest probability with the distribution $\hat{R}_m$. The label chosen by the model m for the sequence X^{test} is modelled by the function $\beta_m(X^{test}) := \prod_{k=1}^l \mathbb{I}_{S_{m,g}}(X_k^{test})$ where $\mathbb{I}_{S_{m,g}}$ denotes the indicator function for the set $S_{m,g}$. If model m flags the sequence X^{test} as benign, then $\beta_m(X^{test}) = 1$. Otherwise, $\beta_m(X^{test}) = 0$.

The various predictions are collected and the label that is chosen by a majority of models is kept as the final prediction. The vote count is defined by $V(X^{test}) := \sum_{m=1}^d \beta_m(X^{test})$. The aggregating detector declares X^{test} benign if $\frac{d}{2} \leq V(X^{test})$, anomalous otherwise.

Theorem 1 (finite- d, l false positive bound). *Assuming that the test sequence X^{test} is benign, i.e. $X_k^{test} \stackrel{i.i.d.}{\sim} P$, there exists t and u in $[0, 1]$ independent from d and ν such that*

$$\mathbb{P}_P \left(V(X^{test}) < \frac{N_{models}}{2} \right) \leq \exp \left(-2N_{models} \left(\nu t + (1 - \nu)u - \frac{1}{2} \right)_+^2 \right).$$

The expressions of t and u and the proof can be found in Appendix A

Numerical analysis In Figure 2, we analyse the role of the parameters ν and d when the other is fixed. As expected, an exponential decrease is observed in

Parameter	Meaning	Illustrative example
K	Finite set of log templates identifiers	$K = \{a, b, c\}$
$\hat{R}_m$	Empirical distribution learned by model m	$\hat{R}_m(a) = 0.5, \hat{R}_m(b) = 0.3, \hat{R}_m(c) = 0.2$
g	Number of templates retained by each model	$g = 2$
$S_{m,g}$	Set of the g most likely templates according to model m	$S_m = \{a, b\}$
X^{test}	Test log sequence	$X^{test} = (a, b, a)$
$\beta_m(X)$	Equals 1 if model m accepts the sequence X . Equals 0 otherwise.	$\beta_m((a, b, a)) = 1$ but $\beta_m((a, c, b)) = 0$
$V(X)$	Number of models accepting the sequence X	If 7 models out of 10 accept the sequence (a, b, a) , then $V((a, b, a)) = 7$

Table 2: Parameters and illustrative example of votes aggregation

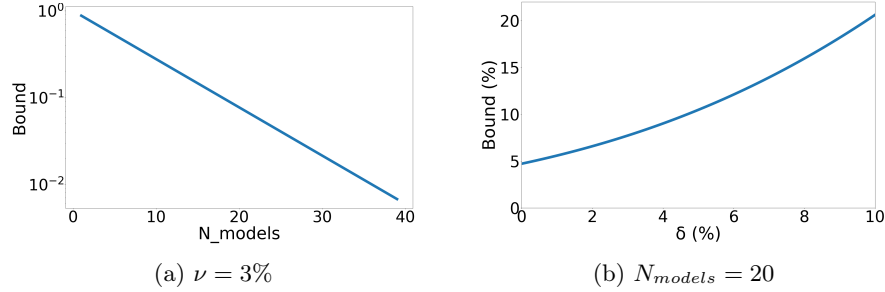


Fig. 2: Bound on the false positive rate with $t = 2 \times 10^{-5}$ and $u = 0.776$.

Figure 2a along with the number of observed sequences d as long as $\nu t + (1 - \nu)u - \frac{1}{2}$ is positive. The probability of false positive is close to 0 when the ratio ν of poisoned models is null and increases as ν rises. Figure 2b shows that the bound grows steadily when ν increases: from 0.05 when $\nu = 0$ to 0.2 when $\nu = 0.1$. With $\nu = 3\%$, $N_{models} = 20$, $t = 2 \times 10^{-5}$ and $u = 0.776$, our bound guarantees a probability of false positives of less than 7.8%.

4.2 Median estimation of the probabilities

We provide robustness guarantees of the composite model when the estimations of the probability distributions are aggregated with the median element-wise. Our result is independent of the poisoning strategy and holds provided that most of the estimations are legitimate and sufficiently accurate. The key idea is that the median of the estimations is always bounded by legitimate estimations.

Hence, if all legitimate estimations are precise, then so is the median of the estimations.

We consider anomaly detection operating on a finite alphabet $K = \{x_1, \dots, x_m\}$. The symbols are ordered by decreasing probability under a reference distribution P , $p_1 > p_2 > \dots > p_m$, $p_i = P(X = x_i)$. A threshold g defines a partition $A_g = \{x_1, \dots, x_g\}$, $B_g = \{x_{g+1}, \dots, x_m\}$. The set A_g contains *benign* symbols and B_g contains *anomalous* symbols. A sequence $X = (X_1, \dots, X_n)$ is declared anomalous if at least one of the X_k belongs to B_g . We provide theoretical guarantees of the robustness of the set A_g under adversarial perturbations. Let assume that the estimators of the distribution P on K are mutually independent and are denoted $(\hat{R}_j)_{j=1, \dots, d}$. We suppose that a fraction δ of the estimators are corrupted. We denote the set of indexes of the corrupted estimators by $\mathcal{C}$ and the set of indexes of legitimate estimators by $\mathcal{L}$. For a set of n real values $\{x_1, \dots, x_n\}$, we define the *median* of the set as the smallest value of this set that is greater than or equal to at least $\lceil \frac{n}{2} \rceil$ values. We assume that, for each logkey $x \in K$, the estimated probabilities $(\hat{R}_j(x) \text{ for } j \in \{1, \dots, d\})$ are almost surely all distinct. This is satisfied e.g., if $(\hat{R}_i)_{i=1, \dots, d}$ have multivariate probability density functions, e.g. Dirichlet distributions. Let $\hat{R}_{med}$ be the distribution proportional to the median of the estimations element-wise: $\hat{R}_{med}(x) = C \times \text{med}(\hat{R}_1(x), \dots, \hat{R}_d(x))$ for $x \in K$ with $C = \frac{1}{\sum_{x \in K} \text{med}(\hat{R}_1(x), \dots, \hat{R}_d(x))}$.

Theorem 2 (Exact recovery of anomaly set). *Suppose that most of the estimators are legitimate i.e. $\delta := \frac{|\mathcal{C}|}{d} \leq \frac{1}{2} - \frac{1}{d}$ and suppose that all legitimate estimators give sufficiently precise estimations of P i.e. $\forall i \in \mathcal{L}, \sum_{x \in K} |\hat{R}_i(x) - P(x)| < \frac{P(x_g) - P(x_{g+1})}{2}$, then $A_g(\hat{R}_{med}) = A_g(P)$.*

Proof. See Appendix B

In the theorem, the hypotheses on $\hat{R}_i$ can be interpreted as the fact that each of these distributions must be close to P . The sum that appears is, apart from a factor, the Total Variation [18] between $\hat{R}_i$ and P which is a distance between two probability distributions.

4.3 Sensitivity of g

A way to influence the robustness of a model is by adjusting the hyperparameter g that controls the size of the set of logkeys that are benign. We herein study the impact of g : we show that if an attacker introduces errors in the estimation of P in order to alter the construction of A_g and B_g , then the attack must exceed a certain threshold to be successful.

Definition 1 (Distributional perturbation). *Let P be a probability distribution on K with probabilities p_i . An adversarial perturbation produces a distribution Q such that for all i , $q_i = p_i + \varepsilon_i$ with constraints $\sum_{i=1}^m \varepsilon_i = 0$, $q_i > 0$. The attack size is measured by $\sum_{i=1}^m |\varepsilon_i|$.*

We study when the anomaly partition (A_g, B_g) is preserved after perturbation.

Theorem 3 (Exact recovery of the anomaly set). *Suppose P and Q satisfy $\sum_{i=1}^m |\varepsilon_i| < p_g - p_{g+1}$. Then $A_g(Q) = A_g(P)$.*

Proof. See Appendix C

From the attacker’s point of view, Theorem 3 means that an effective attack needs a minimal amount of modifications. From the defender’s perspective, Theorem 3 means that, the defender wishing to maximize the robustness of the set $A_g(P)$, must choose the value of g that maximizes $p_g - p_{g+1}$. Recall that p_g is the probability of the rarest benign element and p_{g+1} is the probability of the most likely anomalous element. Hence, in a robust partition, the most likely anomalous element is much rarer than the least likely benign element.

5 Empirical Evaluation

In order to evaluate the performance of our poisoning and mitigation methods, we rely the so-called HDFS dataset [33,34], which contains the logs generated by the Hadoop Distributed File System (HDFS), which is running on 203 nodes, during 2 days. The HDFS dataset consists of 11,175,629 log messages. Each log message has an identifier and a *block id*, based on which logs are grouped into sessions. Among the overall 575,061 sessions, 16,838 sessions (2.93%) are labelled as anomalous, the rest are labelled as benign [33].

Anomaly detector based on DeepLog [6] - the AIDS we implemented includes two types of detectors: *LSTM-based detectors* described in Section 3.2 and *composite detectors* that implement our mitigation method. For the latter, each base model is an *LSTM-based detector* and we hereafter evaluate the different combinations of training set construction method and aggregation method. The dataset was parsed by our AIDS using Drain [10] and provided 28 log templates: our AIDS parses and groups the dataset logs into lists of log templates. Each template is then mapped to a unique integer in $\{0, \dots, |K| - 1\}$ yielding sequences of integers. Given an input sequence, the anomaly detector sequentially processes some sliding windows of size h . Each window is transformed into a matrix concatenating the corresponding one-hot encoded vectors, which is converted into a PyTorch⁵ tensor and provided to the LSTM. LSTM models of our anomaly detectors are implemented with PyTorch and include $L = 2$ layers and a hidden size of $\alpha = 64$. The resulting logits are used as parameters of the softmax function that computes the probability distribution of the log template that follows the currently observed window. During the evaluation, hyper-parameters satisfy: $g = 8$, $h = 8$. Any log sequence whose length $< h + 1$ is discarded and we use the *cross-entropy loss* for training. For each experiment, 5000 sequences are selected from the parsed dataset. 4000 of them are dedicated to the training and 1000

⁵ <https://pytorch.org/>. Last accessed: 2026-06-03

to the test. Among the 4000 sequences used for the training, a proportion δ of them are anomalous sequences, the rest of them are benign sequences. We focus on discreet poisoning attacks and we keep δ inferior to $\delta_{max} = 3\%$. Among the 1000 sequences used for test, 973 sequences are benign and 27 are anomalous. For the burst attacks, $\gamma := \frac{W}{N_{train}} = 0.3$. Each experiment is repeated 10 times and the results are averaged. To facilitate the comparison, the training sets have the same size for the *bagging* and *splitting* methods: $q_b := \lfloor \frac{\text{Training set size}}{\text{Number of base models}} \rfloor$.

In the following, we evaluate the impact of the poisoning on the AIDS (§ 5.1) and on the mitigation strategy (§ 5.2) considering the accuracy (precision, recall and F1-score) and the associated delays.

5.1 Impact of poisoning

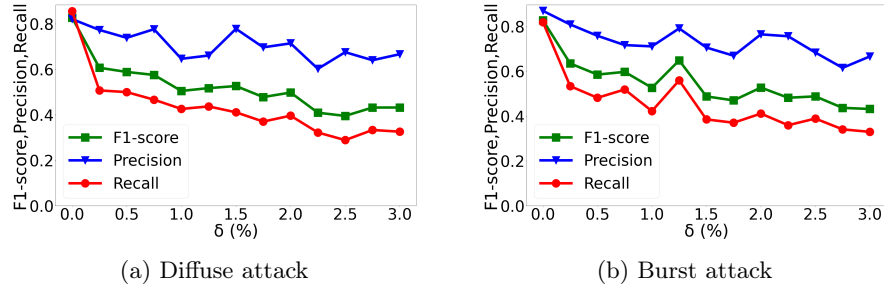


Fig. 3: Impact of the diffuse and burst attacks on the AIDS.

In Figure 3, we analyse the impact of poisoning - expressed as the percentage of sequences that are poisoned in the training set, on the performance of the anomaly detection, considering diffuse (3a) and burst attacks (3b). The impact of the poisoning attacks on the precision, recall and F1-score is quite similar for diffuse and burst attacks. As expected, all the metrics diminish when δ increases. The recall is the most affected by the poisoning: for the two poisoning attacks, a strong decrease ranging from 0.84 with $\delta = 0$ to 0.52 with $\delta = 0.25\%$, is observed. Then, the recall steadily decreases from 0.52 when $\delta = 0.25\%$ to 0.33 when $\delta = 3\%$ for the two poisoning. The higher the proportion of poison in the training set, the less likely the model detects genuine anomalies. As the anomaly detector literally learns anomalous behaviour to be normal, a decrease of the recall is expected. Yet the decrease is remarkable.

5.2 Mitigation method

We analyse in Fig. 4 the effectiveness of our mitigation methods considering a number of models varying from 1 to 23 and 3% of the training set poisoned (i.e. 120 sequences are poisoned). Overall, results indicate that with enough

models, the composite detector achieves good performance despite a poisoned training set. The aggregation method does not seem to impact the effectiveness of the mitigation method. With the *splitting* method, the composite detector performs better with *burst attack* (Fig. 4b, 4f) comparing to *diffuse attack* (Fig. 4a, 4e): less models are required for the same accuracy. The *bagging* and the *splitting* methods have a similar effect with the diffuse attack. Nonetheless, at some point, increasing the number of models ceases to improve performance while the computational overhead keeps improving.

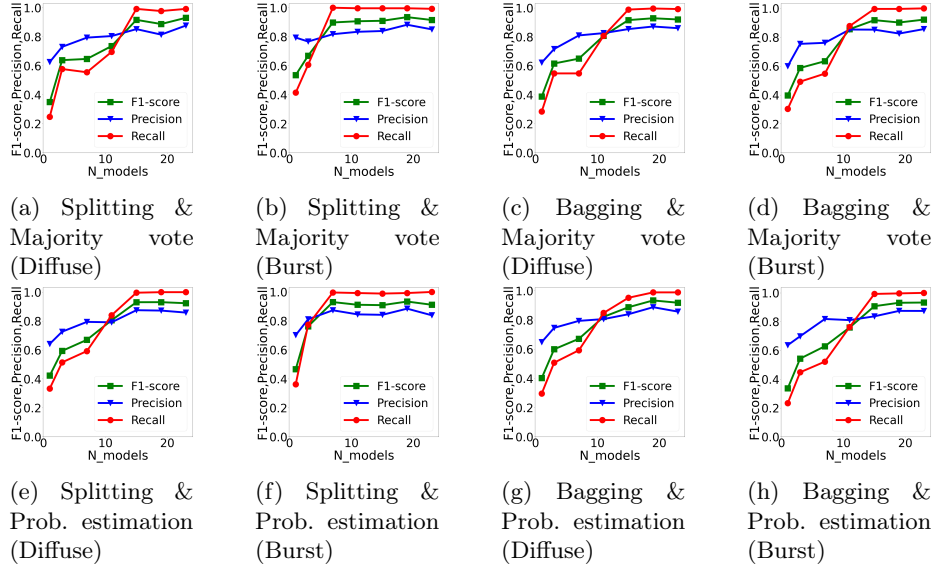


Fig. 4: Effectiveness of the mitigation method with $\delta = 3\%$

We evaluate in Figure 5 the delay associated to the training and inference phases considering the number of models. The duration associated to the inference phase is the mean time required to process a single test sequence. Figure 5a shows that neither the training sets construction method nor the number of model impacts the training duration. This was expected as the number of training samples remains the same. Figure 5b shows that during the inference, the majority vote method is twice as fast than the probability estimation method.

5.3 Robustness of the mitigation methods

When using our mitigation method on a poisoned training set, we expect that some of the base models get corrupted due to the presence of poisoned sequences in the training set. In Figure 6, we determine the number of corrupted base models that our mitigation method can handle. We set $N_{models} = 15$ and we

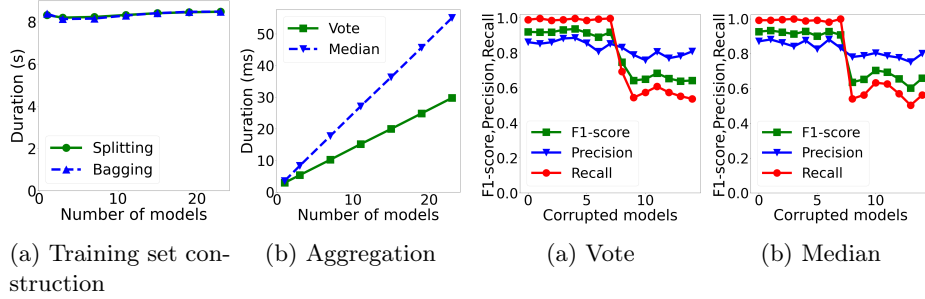


Fig. 5: Impact of the training set construction and aggregation methods

Fig. 6: Impact of corrupted models on the mitigation method. $N_{models} = 15$

use the splitting method to build the different training sets. Each base model is trained on 266 (or 267 for 10 of them) log sequences. We evaluate the performance of our mitigation method when n base models are corrupted with $n \in \{0, \dots, 14\}$. In the experiment, when a base model is corrupted, 20% of its training set, i.e. 53 log sequences, are poisoned, the rest corresponds to benign sequences. Similarly to 5.1, poisoned sequences are sequences labelled as anomalous. If a base model is not corrupted, all its training sequences are benign. Figure 6 shows that the performances of the composite model remain high, regardless of the aggregation method, if less than half of the models are corrupted: the F1-score remains around 0.91 until 8 models are corrupted. We observe two regimes: one regime with less than half of the base models corrupted, and the other regime with more than half of the base models corrupted. In a given regime, the performances barely change. With this method of corruption, the proportion of false alarms is barely affected by the amount of corrupted base models. The values of the recall significantly vary for each regime. The experiments show that as long as most base models are not corrupted, the ability of our mitigation method to alleviate the impact of poisoning, is not affected by the number of corrupted base models. When half of the base models' training sets have a corruption level of 20%, the poisoned sequences represent 10% of all training data. Hence, against an attacker capable of targeting specific base models and poisoning 10% of the whole training set, our mitigation method is unable to alleviate the effect of the poisoning. Using randomness in the construction of the training sets prevents targeted poisoning attacks. Note that bagging intrinsically leverages randomness. Introducing randomness in the splitting method could consist in shuffling the sequences before creating the training sets. The resulting attack would then be similar to a regular diffuse attack. The property for the detector to perform well against burst attacks will then be lost.

6 Conclusion

Logs constitute a vital source of information for detecting potential security issues, relying on the anomalies that indicate security breaches, system faults, and eventually operational failures. Intrusion detection systems can leverage DL tools for efficient log analysis. However, such tools remain very sensitive to poisoning attack: a small fraction of corrupted data can significantly alter the detector’s ability to detect anomalies. In this work, we have proposed and evaluated the performance of a robust anomaly-based intrusion detection system that mitigates log poisoning attacks. Our method consists in training several base models and aggregate their predictions in a robust way. Our evaluations show that the splitting method that partitions the training dataset is highly effective (i.e., characterised by a high F1-score) and that the proposed aggregation methods both support the robust detection. In addition, we provide theoretical analysis to illustrate their effectiveness. Provided enough base models our mitigation method alleviates the effect of our poisoning method. In order to maximise the performance of the robust model, the experimental results suggest to increase the number of base models, provided a suitable method to build the training sets. Nonetheless, increasing the number of base models slows down the anomaly detector during inference. A trade-off is to found in order to guarantee high anomaly detection performance and a suitable process duration for real-time application. Future research will deal with the following points: (i) developing the theoretical analysis, namely the handling of time dependency, (ii) exploring various attacker’s constraints and strategy as well as the use of randomness for defense, (iii) dealing with the trade-off between effectiveness and efficiency and providing automatic ways to determine a suitable number of base models.

References

1. Breiman, L.: Bagging predictors. *Machine Learning* **24**(2), 123–140 (1996)
2. Cheng, H., Xu, D., Yuan, S.: Backdoor attack against log anomaly detection models. In: *the ACM Web Conference*. p. 915–918 (2025)
3. Cinà, et al.: Wild patterns reloaded: A survey of machine learning security against training data poisoning. *ACM Comput. Surv.* **55**(13s) (2023)
4. Dai, J., Chen, C., Li, Y.: A backdoor attack against lstm-based text classification systems. *IEEE Access* **7**, 138872–138878 (2019)
5. Du, M., Jia, R., Song, D.: Robust anomaly detection and backdoor attack detection via differential privacy (2019)
6. Du, M., et al.: Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: *ACM SIGSAC CCS*. pp. 1285–1298 (2017)
7. Ferrara, M.: Data poisoning and artificial intelligence modeling: Theoretical foundations and defensive strategies. In: *2nd Workshop “New frontiers in Big Data and Artificial Intelligence” (BDAI)* (2025)
8. Han, S., et al.: Log-based anomaly detection with robust feature extraction and online learning. *IEEE Transactions on Information Forensics and Security* **16**, 2300–2311 (2021)

9. Hartle III, F., Mancini, S., Kerry, E.: Data poisoning 2018–2025: A systematic review of risks, impacts, and mitigation challenges. *Issues in Information Systems* **25**(4), 433–442 (2025)
10. He, P., et al.: Drain: An online log parsing approach with fixed depth tree. In: *IEEE ICWS*. pp. 33–40 (2017)
11. Herath, J.D., Yang, P., Yan, G.: Real-time evasion attacks against deep learning-based anomaly detection from distributed system logs. In: *11th ACM CODASPY*. p. 29–40 (2021)
12. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**, 1735–1780 (1997)
13. Hong, S., et al.: On the effectiveness of mitigating data poisoning attacks with gradient shaping (2020)
14. Huang, S., et al.: Black-box attacks to log-based anomaly detection. In: *18th CNSM*. pp. 310–316 (2022)
15. Khraisat, A., et al.: Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* **2** (2019)
16. Kloft, M., Laskov, P.: Online anomaly detection under adversarial impact. In: *13th AISTATS*. vol. 9, pp. 405–412 (2010)
17. Koh, P.W., Steinhardt, J., Liang, P.: Stronger data poisoning attacks break data sanitization defenses. *Machine Learning* **111**(1), 1–47 (2022)
18. Levin, D.A., Peres, Y., Wilmer, E.L.: *Markov Chains and Mixing Times*. American Mathematical Society, Providence, RI, 2 edn. (2009)
19. Li, J., Ji, S., Du, T., Li, B., Wang, T.: Textbugger: Generating adversarial text against real-world applications. In: *Proceedings 2019 Network and Distributed System Security Symposium*. NDSS 2019, Internet Society (2019)
20. Li, Y., Jiang, Y., Li, Z., Xia, S.T.: Backdoor learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems* **35**(1), 5–22 (2024)
21. Lu, et al.: Black-box attacks against log anomaly detection with adversarial examples. *Information Sciences* **619**, 249–262 (2023)
22. Miyato, T., Dai, A.M., Goodfellow, I.: Adversarial training methods for semi-supervised text classification (2021)
23. Qi, F., Li, M., Chen, Y., Zhang, Z., Liu, Z., Wang, Y., Sun, M.: Hidden killer: Invisible textual backdoor attacks with syntactic trigger (2021)
24. Rizvi, S., Fordham, E.: *Cyber security breaches survey*. Tech. rep., Department for Science, Innovation and Technology and Home Office (2025)
25. Rubinstein, B.I., et al.: Antidote: understanding and defending against poisoning of anomaly detectors. In: *9th ACM SIGCOMM IMC*. pp. 1–14 (2009)
26. Sagi, O., Rokach, L.: Ensemble learning: A survey. *WIREs Data Mining and Knowledge Discovery* **8**(4) (2018)
27. Samanta, S., Mehta, S.: Towards crafting text adversarial samples (2017)
28. Sun, J., Liu, B., Hong, Y.: Logbug: Generating adversarial system logs in real time. In: *29th ACM CIKM*. p. 2229–2232 (2020)
29. Tan, K., et al.: Multi-stage defense: Enhancing robustness in sequence-based log anomaly detection. In: *IEEE ICC*. pp. 2725–2730 (2024)
30. Venkatesan, S., et al.: Poisoning attacks and data sanitization mitigations for machine learning models in network intrusion detection systems. In: *IEEE MILCOM*. pp. 874–879 (2021)
31. Wu, J., et al.: Aar-log: A robust log anomaly detection method resisting adversarial attacks. *Computer Networks* **269**, 111471 (2025)
32. Xu, C., et al.: Mitigating data poisoning in text classification with differential privacy. In: *ACL EMNLP*. pp. 4348–4356 (2021)

33. Xu, W., et al.: Detecting large-scale system problems by mining console logs. In: ACM SIGOPS 22nd SOSP. p. 117–132 (2009)
34. Zhu, J., et al.: Loghub: A large collection of system log datasets for ai-driven log analytics. In: IEEE 34th ISSRE. pp. 355–366 (2023)

A Proof of Theorem 1

Parameter name	Description	Value
ν	Ratio of poisoned models	0.03
N_{models}	Number of models	20
$m_g(P)$	$\max_{A \subset K, A =g} P(A)$	0.995
ε_1	Upper bound of $\text{TV}(\hat{P}_m, P)$	0.01
l	Sequence length	10
ε_2	Upper bound of $\text{TV}(\hat{Q}_m, Q)$	0.03
Δ	$\text{TV}(P, Q)$	0.3
$m_g(Q)$	$\max_{A \subset K, A =g} Q(A)$	0.7
τ	Minimal ratio of “benign” votes to be declared benign	0.5

Table 3: Reference values and description of the parameters of the upper bound

We rely on a specific notion of distance between probability distributions: given two probability distributions P_1, P_2 on a finite set K , the Total Variation (TV) distance is $\text{TV}(P_1, P_2) := \frac{1}{2} \sum_{a \in K} |P_1(a) - P_2(a)|$. We assume that $\text{TV}(\hat{P}_m, P) \leq \varepsilon_1$, $\text{TV}(\hat{Q}_m, Q) \leq \varepsilon_2$, and we denote $\Delta := \text{TV}(P, Q)$. The descriptions and numerical values of the upper bound’s parameters are summarized in Table 3.

Theorem 4 (finite- d, l false positive bound). *Assuming that the test sequence X^{test} is benign, i.e. $X_k^{test} \stackrel{i.i.d.}{\sim} P$. Define $t := (m_g(Q) - 2\varepsilon_2 - \Delta)_+^l$, $u := (m_g(P) - 2\varepsilon_1)_+^l$ then*

$$\mathbb{P}_P(V(X^{test}) < \tau N_{models}) \leq \exp(-2N_{models}(\nu t + (1 - \nu)u - \tau)_+^2).$$

Proof. We proceed in four steps.

Step 1: conditional law of a single vote. Fix $m \in \{1, \dots, N_{models}\}$ and condition on the random set $S_{m,g} \subset K$. By definition, $\beta_m(X^{test}) = 1 \iff X_k^{test} \in S_{m,g}$ for all $k = 1, \dots, l$. Using the i.i.d. assumption and independence, $\mathbb{P}_P(\beta_m(X^{test}) = 1 \mid S_{m,g}) = P(S_{m,g})^l$. Taking expectation yields $\mathbb{E}_P[\beta_m(X^{test})] = \mathbb{E}[P(S_{m,g})^l]$.

Step 2: clean models. If $m \notin \mathcal{C}$, then $S_{m,g} = S_g(\hat{P}_m)$ and $\text{TV}(\hat{P}_m, P) \leq \varepsilon_1$.

Let $S^* = S_g(P)$. Using total variation,

$$\begin{aligned}
P(S_{m,g}) - m_g(P) &= P(S_{m,g}) - P(S_g(P)) = P(S_{m,g}) - P(S^*) \\
&= \underbrace{P(S_{m,g}) - \hat{P}_m(S_{m,g})}_{\geq -\text{TV}(\hat{P}_m, P)} + \underbrace{\hat{P}_m(S_{m,g}) - \hat{P}_m(S^*)}_{\substack{\geq 0 \\ \text{by definition of } S_{m,g}}} + \underbrace{\hat{P}_m(S^*) - P(S^*)}_{\geq -\text{TV}(\hat{P}_m, P)} \\
&\geq -2 \times \text{TV}(\hat{P}_m, P) \geq -2\varepsilon_1,
\end{aligned}$$

implying $\mathbb{E}_P[\beta_m(X^{test})] \geq u$.

Step 3: contaminated models. If $m' \in \mathcal{C}$, then $S_{m',g} = S_g(\hat{Q}_{m'})$ and $\text{TV}(\hat{Q}_{m'}, Q) \leq \varepsilon_2$. Arguing identically, we have that $Q(S_{m',g}) \geq m_g(Q) - 2\varepsilon_2$, and transferring to P , $P(S_{m',g}) \geq m_g(Q) - 2\varepsilon_2 - \Delta$ implying that $\mathbb{E}_P[\beta_{m'}(X^{test})] \geq t$.

Step 4: concentration. By linearity,

$$\mathbb{E}_P[V(X^{test})] \geq N_{models}(\nu t + (1 - \nu)u).$$

– If $\nu t + (1 - \nu)u \leq \tau$: $(\mu_P - \tau d)_+ = 0$

$$\mathbb{P}_P(V(X^{test}) < \tau d) \leq 1 = \exp(-2N_{models}(\nu t + (1 - \nu)u - \tau)_+^2) \text{ is immediate}$$

– If $\tau \leq \nu t + (1 - \nu)u$: $(\nu t + (1 - \nu)u - \tau)_+ = \nu t + (1 - \nu)u - \tau \geq 0$

Since the votes are independent and bounded within $[0, 1]$, Hoeffding's inequality gives for all $\alpha \geq 0$

$$\mathbb{P}_P(V(X^{test}) - \mathbb{E}_P[V(X^{test})] < -\alpha) \leq \exp\left(-\frac{2\alpha^2}{N_{models}}\right)$$

For $\alpha = N_{models}(\nu t + (1 - \nu)u - \tau)$,

$$\begin{aligned}
\mathbb{P}_P(V(X^{test}) - \mathbb{E}_P[V(X^{test})] < -N_{models}(\nu t + (1 - \nu)u - \tau)) \\
\leq \exp(-2N_{models}(\nu t + (1 - \nu)u - \tau)^2)
\end{aligned}$$

As $\mathbb{E}_P[V(X^{test})] \geq N_{models}(\nu t + (1 - \nu)u)$,

$$V(X^{test}) - \mathbb{E}_P[V(X^{test})] < -N_{models}(\nu t + (1 - \nu)u - \tau)$$

Let us define

$$\begin{aligned}
C_1 &:= \{V(X^{test}) < \tau N_{models}\} \\
C_2 &:= \{V(X^{test}) - \mathbb{E}_P[V(X^{test})] < -N_{models}(\nu t + (1 - \nu)u - \tau)\}.
\end{aligned}$$

We have that $C_1 \subset C_2$ implying $\mathbb{P}_P(C_1) \leq \mathbb{P}_P(C_2)$. Using the previous inequalities we obtain:

$$\mathbb{P}_P(V(X^{test}) < \tau N_{models}) \leq \exp(-2N_{models}(\nu t + (1 - \nu)u - \tau)_+^2)$$

which concludes the proof. $\square$

B Proof of Theorem 2

Proof. This proof will be done in 3 steps. First we will show that for each element $x \in K$, there are always legitimate estimations greater than and lesser than the median of the estimation of the probability. Second, we will present a bound of the difference between the median of the estimations of the probability of $x \in K$ and $P(x)$. And third, we will show that the previous bound guarantees the recovery of the anomaly set.

Step 1 (There are legitimate estimations greater than and lesser than the median of the estimations): This will be a proof by contradiction. Let $j \in \{1, \dots, m\}$. Let $\lambda_j := \text{med}(\hat{R}_1(x_j), \dots, \hat{R}_d(x_j))$. Let's suppose that for all $i \in \mathcal{L}$, $\hat{R}_i(x_j) \leq \lambda_j$. Then

$$\frac{d}{2} + 1 \leq d(1 - \delta) \leq d(1 - \delta) + \sum_{i' \in \mathcal{C}} \mathbb{I}_{(-\infty, \lambda_j]}(\hat{R}_{i'}(x_j)) = \sum_{i=1}^d \mathbb{I}_{(-\infty, \lambda_j]}(\hat{R}_i(x_j)),$$

Which contradicts the definition of λ_j as the $(\hat{R}_i(x_j))_{i=1, \dots, d}$ are all distinct.

Now, let's suppose that for all $i \in \mathcal{L}$, $\lambda_j < \hat{R}_i(x_j)$. Then

$$\sum_{i=1}^d \mathbb{I}_{(-\infty, \lambda_j]}(\hat{R}_i(x_j)) = \sum_{i' \in \mathcal{C}} \mathbb{I}_{(-\infty, \lambda_j]}(\hat{R}_{i'}(x_j)) \leq |\mathcal{C}| = \delta d \leq \frac{d}{2} - 1$$

Which contradicts the definition of λ_j . Hence for all $j \in \{1, \dots, m\}$ there exists i_j^+ and i_j^- both in $\mathcal{L}$ such that $\hat{R}_{i_j^-}(x_j) \leq \lambda_j < \hat{R}_{i_j^+}(x_j)$.

Step 2 (Bound on the error): Let $j \in \{1, \dots, m\}$. Note that

$$\begin{aligned} -2 \times TV(\hat{R}_{i_j^-}, P) &= - \sum_{j'=1}^m \left| \hat{R}_{i_j^-}(x_{j'}) - P(x_{j'}) \right| \leq - \left| \hat{R}_{i_j^-}(x_j) - P(x_j) \right| \\ &\leq \hat{R}_{i_j^-}(x_j) - P(x_j) \leq \lambda_j - P(x_j) \end{aligned}$$

By hypothesis, $TV(\hat{R}_{i_j^-}, P) < \frac{P(x_g) - P(x_{g+1})}{4}$. Then $-\frac{P(x_g) - P(x_{g+1})}{2} < \lambda_j - P(x_j)$. Similarly, $\lambda_j - P(x_j) < \frac{P(x_g) - P(x_{g+1})}{2}$.

Step 3 (Final inequality): Note that $A_g(\hat{R}_{med}) = A_g(P) \Leftrightarrow \forall k \in \{1, \dots, g\}, l \in \{g+1, \dots, m\}, \hat{R}_{med}(x_l) < \hat{R}_{med}(x_k)$. Let $k \in \{1, \dots, g\}$ and $l \in \{g+1, \dots, m\}$ such that

$$\begin{aligned} \hat{R}_{med}(x_l) < \hat{R}_{med}(x_k) &\Leftrightarrow C \times \text{med}(\hat{R}_1(x_l), \dots, \hat{R}_d(x_l)) < C \times \text{med}(\hat{R}_1(x_k), \dots, \hat{R}_d(x_k)) \\ &\Leftrightarrow \lambda_l < \lambda_k \Leftrightarrow 0 < (\lambda_k - P(x_k)) + P(x_k) - P(x_l) + (P(x_l) - \lambda_l). \end{aligned}$$

With the previous step, we have $-\frac{P(x_g)-P(x_{g+1})}{2} < P(x_l)-\lambda_l$ and $-\frac{P(x_g)-P(x_{g+1})}{2} < \lambda_k - P(x_k)$.

As the element of K are ordered by decreasing likelihood, and as $k \in \{1, \dots, g\}$ and $l \in \{g+1, \dots, m\}$, $P(x_g) \leq P(x_k)$ and $P(x_l) \leq P(x_{g+1})$. Then, $P(x_g) - P(x_{g+1}) \leq P(x_k) - P(x_l)$. From these inequalities, we obtain

$$\begin{aligned} & -\frac{P(x_g) - P(x_{g+1})}{2} + P(x_g) - P(x_{g+1}) - \frac{P(x_g) - P(x_{g+1})}{2} \\ & < (\lambda_k - P(x_k)) + P(x_k) - P(x_l) + (P(x_l) - \lambda_l), \end{aligned}$$

implying that $0 < \lambda_k - \lambda_l$, which concludes the proof. $\square$

C Proof of Theorem 3

Proof. Write $q_i = p_i + \varepsilon_i$ where $\varepsilon_i = q_i - p_i$. By assumption $\sum_{i=1}^m |\varepsilon_i| < \Delta_g$. To prove that the top- g set is preserved, it suffices to show that every index inside the set $\{1, \dots, g\}$ remains larger than every index outside it. That is, we prove that for all $i \leq g$ and $j > g$, $q_i > q_j$. Let $i \leq g$ and $j > g$. Since P is ordered, $p_i \geq p_g > p_{g+1} \geq p_j$, implying that $p_i - p_j \geq p_g - p_{g+1} = \Delta_g$. Now $q_i - q_j = (p_i - p_j) + (\varepsilon_i - \varepsilon_j)$. Using the triangle inequality, $\varepsilon_i - \varepsilon_j \geq -(|\varepsilon_i| + |\varepsilon_j|)$. Therefore

$$q_i - q_j \geq (p_i - p_j) - (|\varepsilon_i| + |\varepsilon_j|) \geq \Delta_g - (|\varepsilon_i| + |\varepsilon_j|).$$

Since $|\varepsilon_i| + |\varepsilon_j| \leq \sum_{k=1}^m |\varepsilon_k| < \Delta_g$, we obtain $q_i - q_j > 0$. Thus $q_i > q_j$ for every $i \leq g$ and $j > g$. Consequently $\min_{i \leq g} q_i > \max_{j > g} q_j$, which implies that the g largest components of Q are exactly those with indices $1, \dots, g$. Hence $A_g(Q) = \{1, \dots, g\} = A_g(P)$.